

Documentación de código

Comentarios y Javadoc

Idea clave — ¿para qué sirve documentar?

La documentación sirve para que otra persona — o tú mismo dentro de dos meses — pueda entender qué hace el código, cómo se usa y por qué se tomaron ciertas decisiones.

En Java hay dos formas que se complementan:

	Comentario <code>//</code> o <code>/* */</code>	Javadoc <code>/** */</code>
Para qué	Aclarar una decisión o un caso especial	Describir clases y métodos públicos
Quién lo lee	El desarrollador que toca ese fichero	Cualquiera que use la clase desde fuera
Lo genera el IDE	No	Sí — se puede exportar a HTML
Cuándo usarlo	Cuando el por qué no es obvio	En toda la API pública del proyecto

¿Por qué documentar?

Documentar no es solo ser ordenado. Tiene consecuencias directas en el día a día:

Trabajo en equipo

Sin documentación, cada nuevo compañero tiene que leer el código entero para entender qué hace cada parte.

Ahorra tiempo

Un comentario de una línea puede evitar 20 minutos de análisis a quien retoma el código.

Evita errores

Las reglas de negocio y las restricciones quedan explícitas, no enterradas en la lógica.

Facilita el mantenimiento

Cuando algo falla, saber por qué está así acelera el diagnóstico.

3.1 Comentarios: cuándo sí y cuándo no

Un buen comentario explica algo que el código no puede decir por sí solo. Pregúntate: ¿aporta algo que el nombre del método no dice ya?

✓ Comentarios útiles

```
// IVA reducido por normativa
// (productos básicos)
double total = precio * IVA_REDUCIDO;

// Bloqueado: no permitimos login
// aunque la contraseña sea correcta
if (user.isBlocked())
    return LoginResult.BLOCKED;
```

✗ Comentarios que sobran

```
// Incrementa i
i++;

// Devuelve el total
return total;
```

💡 Regla práctica: si el comentario solo repite lo que ya dice el nombre, en vez de comentar, mejora el nombre.

¿Qué documentar primero en un proyecto?

No todo necesita el mismo nivel de documentación. Ordena el esfuerzo por impacto real:

1



README (nivel proyecto)

Qué es el proyecto, cómo ejecutarlo, cómo pasar los tests. Lo primero que ve cualquiera que llega al repositorio.

2



Puntos de entrada

Métodos públicos, controladores, endpoints. Lo que otros van a llamar sin ver la implementación.

3



Reglas de negocio

Validaciones, descuentos, permisos, casos especiales. Lo que no es evidente leyendo el código.

4



Decisiones no obvias

"Esto está así porque X" — lo que alguien podría cambiar sin darse cuenta de que rompe algo.

El README: la primera documentación del proyecto

README.md es lo primero que ve cualquiera que llega a un proyecto. No hace falta que sea largo, pero sí útil.

Estructura mínima que funciona:

```
# Nombre del proyecto

Descripción breve: qué hace y para qué sirve.

## Cómo ejecutar
1. Abre el proyecto en IntelliJ.
2. Ejecuta Main.java (punto de entrada).

## Cómo pasar los tests
Desde IntelliJ: clic derecho en test/ → Run All Tests
O desde terminal: mvn test

## Notas
- Requiere Java 17 o superior.
```

💡 Si alguien que no conoce el proyecto puede ejecutarlo en 5 minutos leyendo solo el README, el README es bueno.

3.2 Javadoc: qué es y para qué sirve

Javadoc es el formato estándar de Java para documentar clases y métodos públicos. Al pasar el ratón sobre un método en IntelliJ, muestra la documentación directamente. Se escribe con `/** ... */` y se puede exportar a HTML.

Etiquetas más habituales — usa solo las que aporten información real:

Etiqueta	Dónde	Para qué
@author	clase	autor o responsable
@version	clase	versión del componente
@since	clase	desde qué versión existe
@param	método	describe cada parámetro de entrada
@return	método	qué devuelve (y en qué casos)
@throws	método	cuándo lanza una excepción
@deprecated	cualquiera	si ya no se recomienda usar, y qué usar en su lugar

⚠ Si **@version** no se usa en tu proyecto, no lo pongas por decorar. Un Javadoc con etiquetas vacías es peor que no tenerlo.

3.2 Javadoc — ejemplo en una clase

Documenta la responsabilidad de la clase, sus reglas y cómo encaja en el proyecto

```
/**
 * Servicio de cálculo de precios para pedidos.
 * Aplica reglas simples de descuento y validaciones básicas.
 *
 * Uso típico: se llama desde la capa de servicio
 *             para obtener el total a pagar.
 *
 * @author Aitor
 * @version 1.0
 * @since 2026-02
 */
public class PriceService { ... }
```

Qué vale la pena documentar en una clase:

- Qué responsabilidad tiene (qué hace y qué no).
- Reglas importantes que aplica.
- Cómo se usa en el proyecto (una frase).

3.2 Javadoc — ejemplo en un método

Documenta qué hace, qué espera de entrada y qué devuelve

```
/**
 * Calcula el precio final de un pedido.
 *
 * @param unitPrice  precio por unidad (debe ser >= 0)
 * @param units      cantidad de unidades (debe ser >= 0)
 * @param premium    indica si se aplica descuento premium
 * @return precio final calculado (nunca negativo)
 * @throws IllegalArgumentException si unitPrice o units son negativos
 */
public static double calculateTotal(
    double unitPrice, int units, boolean premium) { ... }
```

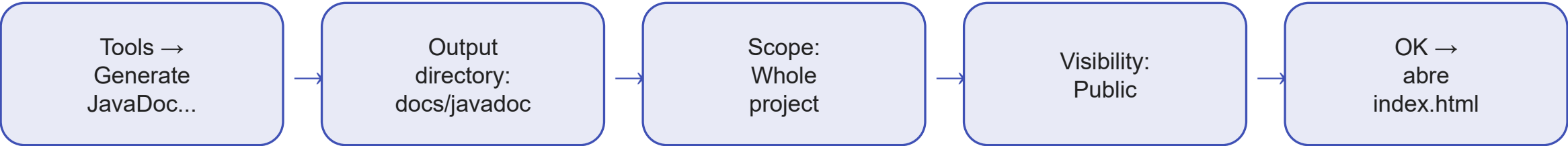
Cuándo NO usar Javadoc

- Métodos privados muy cortos y con nombre claro.
- Código que cambia frecuentemente (la doc se queda desactualizada).
- Getters y setters triviales sin lógica especial.

3.3 Generar documentación HTML desde IntelliJ

Una de las ventajas de Javadoc: puedes generar una web HTML con toda la documentación del proyecto directamente desde IntelliJ, sin instalar nada extra. La página principal del resultado es index.html.

Pasos en IntelliJ:



Problemas típicos y soluciones rápidas

Problema	Solución
Tildes o ñ mal generadas	Revisa que el proyecto usa UTF-8 como encoding
Faltan clases o paquetes	Comprueba el Scope — puede que no estés generando todo el proyecto
Sale demasiado contenido	Reduce visibilidad a Public o limita a un solo paquete